

Open in app ↗

≡ Medium

🔍 Search

✍ Write



Apply now to speak at Medium Day 2026



[aCMM]: Agent Capability Maturity Model - A Unified Framework for Evaluating Modern AI Agents



Suresh Chande · 14 min read · Mar 30, 2026



Agent Capability Maturity Model

A Framework for Evaluating Intelligent Agent Systems and Ecosystems



The term *AI* (artificial intelligence) Agent once referred to a narrow class of autonomous software programs defined by autonomy, reactivity, proactivity, and social ability. Classic frameworks such as JAFIMA (Java Application Framework for Intelligent & Mobile Agents)[1] formalized these ideas through layered architectures for perception, reasoning, and action.



Evaluating Modern AI: Inside the Agent Capability Maturity Model (aCMM)

Preview Mar 30 · Entering the new and bold AI Age of Agents

Save on Spotify

Listen to my Podcast instead [54 Minutes]



Evaluating Modern AI: Why We Need the Agent Capa...

Entering the new and bold AI Age of Agents



See a short video instead [7:36 Minutes]

Today, the label Agent applies to an astonishingly diverse ecosystem:

- **Large language model (LLM)-driven digital Agents** that reason in natural language and orchestrate tools
- **Embodied robotic Agents** navigating physical environments under strict safety constraints
- **Multi-Agent systems** coordinating fleets of drones, vehicles, or specialized digital workers
- **Agentic organizations** where agents manage other agents

This conceptual expansion has outpaced our vocabulary. In practice, teams use the same word — *agent* — to describe systems with radically different

capabilities, risks, and operating constraints. That mismatch is exactly why we need a framework that is concrete enough for engineering and governance, but broad enough to cover both software and robots.

The Problem: No Shared Taxonomy, No Shared Understanding

This ambiguity is not just academic; it turns into real friction the moment a team tries to evaluate an agent for production use. People end up comparing different things under the same label — an LLM workflow assistant, a warehouse robot, and a compliance-controlled copilot can all be called agents, even though they operate under very different constraints. The result is confusion about what is actually being claimed and what evidence is required.

Simple example: In one meeting, a vendor shows a flashy demo and calls their product an “autonomous agent.” Procurement hears “fewer headcount hours,” engineering hears “new failure modes to test,” and compliance hears “new audit requirements.” Without a shared way to describe capabilities (autonomy, tool use, memory, embodiment, and alignment), the discussion collapses into weak proxies like model size or a one-off demo, instead of an evidence-based comparison.

Scenario: Imagine two teams in the same company preparing for a governance review. Team A built a large language model (LLM)-based site reliability engineering (SRE) copilot that can summarize alerts, query observability logs, propose remediation steps, and open ServiceNow incidents. Team B is rolling out **autonomous mobile robots (AMRs)** warehouse robots that navigate facilities and move totes between stations without continuous human control. In the review, both teams describe their system as “highly autonomous.” Team A means the copilot can complete a ticket workflow without constant prompting; Team B means the robot can

navigate intersections, yield to people, and recover from blocked aisles. Those claims sound similar but imply different failure modes, test strategies, and safety bars. Without a shared vocabulary, leadership can easily apply the wrong controls — for example, demanding International Organization for Standardization (ISO)-style safety evidence for a digital copilot, or approving broader physical autonomy without requiring geofencing, emergency stops (e-stops), and incident replay tooling. **This is why a single linear maturity ladder breaks down:** the “right” target is a balanced profile, not a single number.

Traditional maturity models tend to force agents into a single linear ladder[5,6,7,8]. Modern agent systems rarely evolve that way: they may add tool orchestration without improving learning, gain long-term memory while keeping autonomy capped for safety, or increase embodiment while becoming *less* adaptive to satisfy certification. What we need is a multidimensional profile that separates capability from constraint and makes those trade-offs explicit.

The rest of this article introduces a simple way to make those conversations precise: describe any agent as a profile across eight dimensions, score it on evidence, and use that profile to guide design, procurement, and governance decisions.

A Unified Agent Capability Framework

This article proposes a **unified, eight-dimension capability model** that can score any agent — purely digital, embodied, or hybrid — on a **0–5 scale**. The goal is not to declare a single winner, but to produce a repeatable *capability fingerprint* that helps you reason about design intent, operational risk, and readiness for deployment.

First, we define the dimensions briefly. Then, we go deeper with practical guidance for scoring each one from observable behavior and operational evidence.

The Eight Dimensions (What They Measure in Practice)

Dimension	Plain-English interpretation (what you would observe)
Autonomy	How much the agent can initiate, sequence, and complete work without human intervention, including whether it can choose subgoals vs. merely execute instructions.
Reasoning & Planning	Whether it can form multi-step plans, revise them when reality changes, and justify choices, not just pattern-match answers.
Learning & Adaptation	How (and whether) behavior improves over time, from static rules to controlled feedback loops to online learning in safe regimes.
Memory & Context	How it maintains short-term state and long-term knowledge, including retrieval, personalization boundaries, and context-window management.
Tool Use & Integration	How reliably it selects and orchestrates external tools (APIs, software, devices), including error handling, permissions, and rollback.
Collaboration & Social Ability	How it coordinates with humans and other agents, asking clarifying questions, handing off tasks, negotiating interfaces, and preventing cascading failures.
Embodiment	Whether it senses and acts in the physical world, and at what fidelity (navigation, manipulation, dexterity, safety envelopes).
Domain Alignment	How well it complies with domain constraints (policy, regulation, safety cases), and whether it produces auditable evidence of compliant operation.

Scoring each dimension independently produces a **capability fingerprint** you can visualize (for example, as a radar chart) and track across releases as a **Capability Resume**. Two agents can look equally “advanced” in a demo yet have opposite fingerprints. For example, a hospital documentation copilot may score high in Domain Alignment and Collaboration but intentionally

low in Autonomy and Learning, while a research automation agent may prioritize Autonomy, Tool Use, and Learning and accept tighter sandboxing as its safety strategy.

Deep Dive: Capability Dimensions

The table above is the short definition set. In this section, each dimension is expanded into an observable gradient from Level 0 (absent) to Level 5 (state of the art). The scoring rule is simple: you should be able to justify a score with tests, logs, and runtime behavior — not just a demo or a model card.

Level 0 (Absent)		Level 3 (Intermediate)	Level 5 (Advanced)
Autonomy	Agent only acts when explicitly told; no initiative.	Can execute multi-step tasks semi-independently, but within a narrow scope or under human supervision.	Fully self-directed in its domain; sets and pursues goals without human input, reliably and safely.
Reasoning & Planning	No real reasoning; follows hard-coded rules or simple reactions.	Can form basic multi-step plans or handle moderately complex decisions with some support.	Sophisticated strategic reasoning; dynamic planning and problem solving in open-ended scenarios.
Learning & Adaptation	Fixed behavior; does not learn or adapt after deployment.	Learns from feedback in limited ways (e.g. tuning parameters or remembering recent corrections).	Continuously self-improves in real time; autonomously updates its knowledge and strategies.
Memory & Context	Stateless; forgets inputs immediately; no context awareness.	Remembers context within a task or short time frame; limited long-term memory.	Extensive long-term memory and contextual awareness; draws on vast history of interactions and data.
Tool Use & Integration	No external interactions; cannot use tools or services.	Uses a set of predefined tools or APIs when instructed.	Seamlessly integrates with numerous tools and systems; autonomously finds or creates new tools as needed.
Collaboration	Isolated; no concept of other agents or teamwork.	Can interact with others in a limited way (e.g. ask for help, share simple info).	Coordinates effectively in teams of agents and with humans; negotiates, delegates tasks, and collaborates on complex objectives.
Embodiment	No physical presence; cannot move or act in the physical world.	Semi-autonomous operation in structured environments (e.g. follows pre-mapped routes, basic object handling).	Human-level (or better) physical mobility and manipulation; navigates unstructured environments and performs complex physical tasks, safely and reliably.
Domain Alignment	Not aligned with domain specific requirements, would violate key regulations or norms.	Partially domain-aware; meets some requirements but needs human oversight for compliance or safety critical decisions.	Fully compliant with domain standards regulations, and best practices; trusted to operate in domain without special restrictions.

Autonomy

Autonomy measures how far the system can move from *being driven* to *being self-directed*. At low levels, it only reacts to explicit triggers (a button click, a webhook) and follows pre-authored flows. At higher levels, it can decompose goals into subgoals, schedule work, request missing information, and decide when to stop. **Scenario:** an information technology (IT) ops agent at Autonomy 2 can restart services when a specific alert fires; at Autonomy 4 it can detect a slow-burn incident, collect evidence, open a ticket with a proposed fix, and page the on-call, but still respects a human approval gate for risky actions.

Reasoning & Planning

Reasoning & Planning is about structured problem solving under uncertainty: forming a plan, validating assumptions, and repairing the plan when the environment changes. A Level 1 system can produce plausible next steps but struggles to maintain coherence across a long chain. A Level 5 system can branch, compare alternatives, and select strategies that optimize constraints like cost, time, or safety. **Scenario:** a procurement agent asked to reduce cloud spending by 15% might propose obvious rightsizing at Level 2; at Level 4 it can generate a staged plan (inventory → measure → simulate → execute), run impact analysis, and surface trade-offs such as latency risk or reserved-instance lock-in.

Learning & Adaptation

Learning & Adaptation describes whether the agent improves from experience and *how safely* it does so. Early levels are static: behavior changes only when humans redeploy code or a new model. Mid-level systems incorporate feedback loops (thumbs-up/down, human review queues) to tune prompts, retrieval, or policies. High levels use controlled online

learning or reinforcement learning within sandboxes, with drift detection and rollback. **Scenario:** a customer-support agent might learn that certain refund requests correlate with fraud signals, but only after those signals are validated and the policy is updated. *Note: In regulated domains, learning is often intentionally capped to preserve determinism, reproducibility, and certification artifacts.*

Memory & Context

Memory & Context is the difference between an agent that converses and one that *remembers enough to be useful tomorrow*. At low levels, the agent is effectively stateless and can only work within the current prompt. Higher levels add session state, task state, and durable memory with retrieval (often via embeddings) plus explicit controls for retention and privacy. **Scenario:** an enterprise assistant with Level 4 memory can remember project-specific conventions (naming, owners, timelines), retrieve the latest decision from meeting notes, and avoid re-asking already-answered questions while still honoring expiration, redaction, and least-privilege access.

Tool Use & Integration

Tool Use & Integration measures whether the agent can act through reliable interfaces and whether it treats tools as first-class, failure-prone systems. Low-level agents call a single application programming interface (API) with hard-coded parameters. Higher-level agents choose among tools, sequence them, interpret outputs, and recover from partial failures (timeouts, permission errors, inconsistent data). **Scenario:** a finance close agent might reconcile invoices by pulling data from enterprise resource planning (ERP), matching against contracts, and generating exceptions. A Level 5 tool user

not only performs the workflow, but also validates invariants (totals balance), writes an audit trail, and rolls back changes if downstream checks fail.

Collaboration & Social Ability

Collaboration & Social Ability covers coordination protocols: how the agent communicates intent, requests help, delegates, and avoids stepping on other actors. In multi-agent settings this includes role specialization, negotiation, and conflict resolution. **Scenario:** in a software delivery pipeline, one agent might triage issues, another writes a fix, and a third runs tests and prepares a release note. A mature collaborative layer defines contracts (inputs/outputs), escalation rules, and shared memory so that agents do not create circular work or duplicate effort.

Embodiment

Embodiment distinguishes agents that act in pixels from agents that act in atoms. It includes perception (sensor fusion), localization, navigation, manipulation, and safe interaction with people and infrastructure. A high-embodiment agent is rarely “fully autonomous” in the everyday sense; it is bounded by an operational design domain, redundancy, and a safety case. **Scenario:** a warehouse AMR may navigate aisles at Level 4 embodiment but still require strict geofencing, conservative speed limits near people, and a certified e-stop pathway; a field-inspection quadruped might trade learning for stability, so it behaves identically across approved firmware versions and produces consistent incident replays.

Domain Alignment

Domain Alignment is not an extra; it is the constraint layer that determines whether a capability is deployable. It covers policy compliance, regulatory fitness, safety engineering, privacy, and auditability. At low levels, an agent may be impressive in a lab but unusable in production because no one can prove what it did, why it did, or how to contain failures. At high levels, it operates within explicit boundaries, produces evidence (logs, provenance, test results), and supports certification or internal assurance. **Scenario:** a healthcare documentation agent may score high on language ability, but without Health Insurance Portability and Accountability Act (HIPAA)-compliant access controls, data minimization, and traceable citations back to the patient record, it cannot cross the deployment threshold regardless of its reasoning score.

Why a Numeric Scale Matters

A 0–5 scale is useful because it is both **granular** and **governable**: it lets teams distinguish between “the demo worked once” and “this is production reliable,” it supports future extension as new capability classes emerge, it enables comparison across fundamentally different agent types, and it gives risk and compliance functions explicit triggers for controls (for example, requiring stronger approvals, monitoring, and rollback as Autonomy increases).

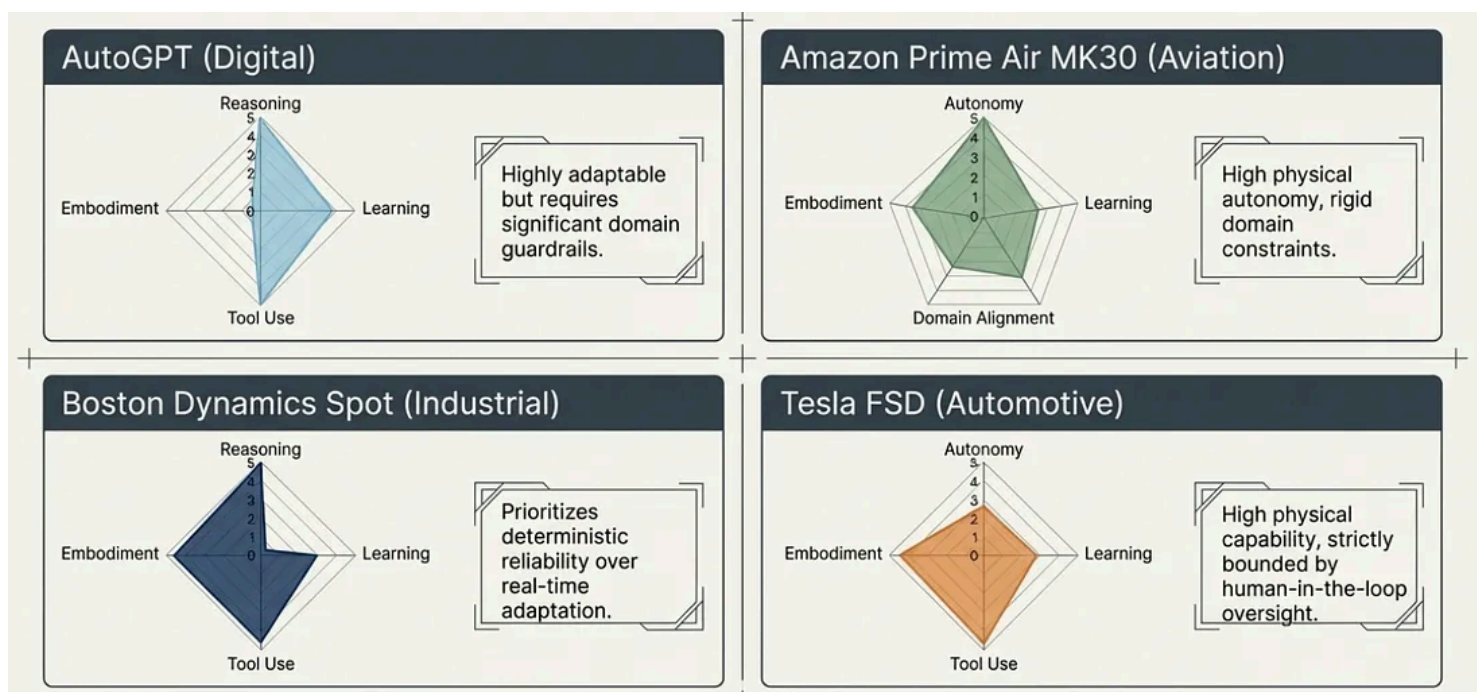
Once you can score dimensions consistently, the conversation changes: you can debate trade-offs (for example, higher autonomy versus stronger controls) using the same measurement language across teams and vendors.

Agent Capability Profiles for four categories of agents

A unified capability model allows radically different agents — digital, embodied, or hybrid — to be evaluated within a single, coherent structure.

By plotting each agent across the eight dimensions, organizations can compare strengths, identify gaps, and understand how design choices reflect domain constraints.

The next section shows how four well-known systems tend to cluster into different capability patterns when viewed through the same eight-dimension lens.



AutoGPT (Digital LLM Agent)

AutoGPT is a representative digital LLM agent: it is strongest when the environment is *software-defined* (documents, tickets, repositories) and when actions can be validated and rolled back. In the framework, it typically scores high on **Reasoning & Planning** and **Tool Use**, has no **Embodiment**, and its **Domain Alignment** depends on the surrounding system design: identity and permissions, scoped tool access, audit logging, and approval workflows for high-impact actions. In practice, it can automate routine knowledge work (triage, drafting, classification, basic workflow execution),

but it should be constrained in regulated workflows unless it can produce traceable evidence for why it acted and what sources it used.

Tesla Full Self-Driving (FSD)[4]

Tesla Full Self-Driving illustrates a capability pattern that is common in safety-critical embodied agents: very high **Embodiment** (perception, localization, control) and substantial system-level **Tool Use** (sensor fusion stacks, mapping, planning modules), but autonomy that is *operationally constrained*. Even if the system can handle many driving situations, the effective autonomy level is bound by supervision requirements and the limits of its operational design domain. Its standout feature in this framework is how **Domain Alignment** is inseparable from capability: safety cases, monitoring, and human takeover protocols define what can be shipped. This is for illustrative purposes, Tesla FSD it is not explicitly called an Agent but embodies qualities of an embodied AI Agent.

Amazon Prime Air Drone (MK30)[3]

Amazon's Prime Air drone (e.g., MK30) is a strong example of high **Autonomy** plus high **Embodiment** where the center of gravity is **Domain Alignment**. Beyond navigation and obstacle avoidance, the differentiator is certification-driven engineering: geofencing, failsafe, flight corridor constraints, remote supervision, and documented compliance. The result is an agent optimized for a narrow but valuable mission profile pattern the model makes easy to compare against broader, less constrained agents.

Boston Dynamics Spot[2]

Boston Dynamics Spot is often misunderstood as an “AI brain” rather than a highly engineered mobility platform. In this model, it tends to score very high on **Embodiment** and physical reliability, with **Reasoning** that is frequently more deterministic than LLM-style planning. That is a feature, not a limitation: many industrial deployments value repeatability, graceful degradation, and safety protocols over continual adaptation. Spot’s profile highlights a key lesson of the framework: the most advanced agent is not always the most adaptive one; it is the one best aligned to its operating domain.

The point is not that one profile is universally better. The point is that the framework makes the trade-offs explicit — so you can choose the right agent pattern for your operating domain and governance posture.

Agent Evolution and “Capability Resumes”

Agents evolve across versions as new models, tools, and integrations are added. The framework supports temporal tracking through a **Capability Resume**.

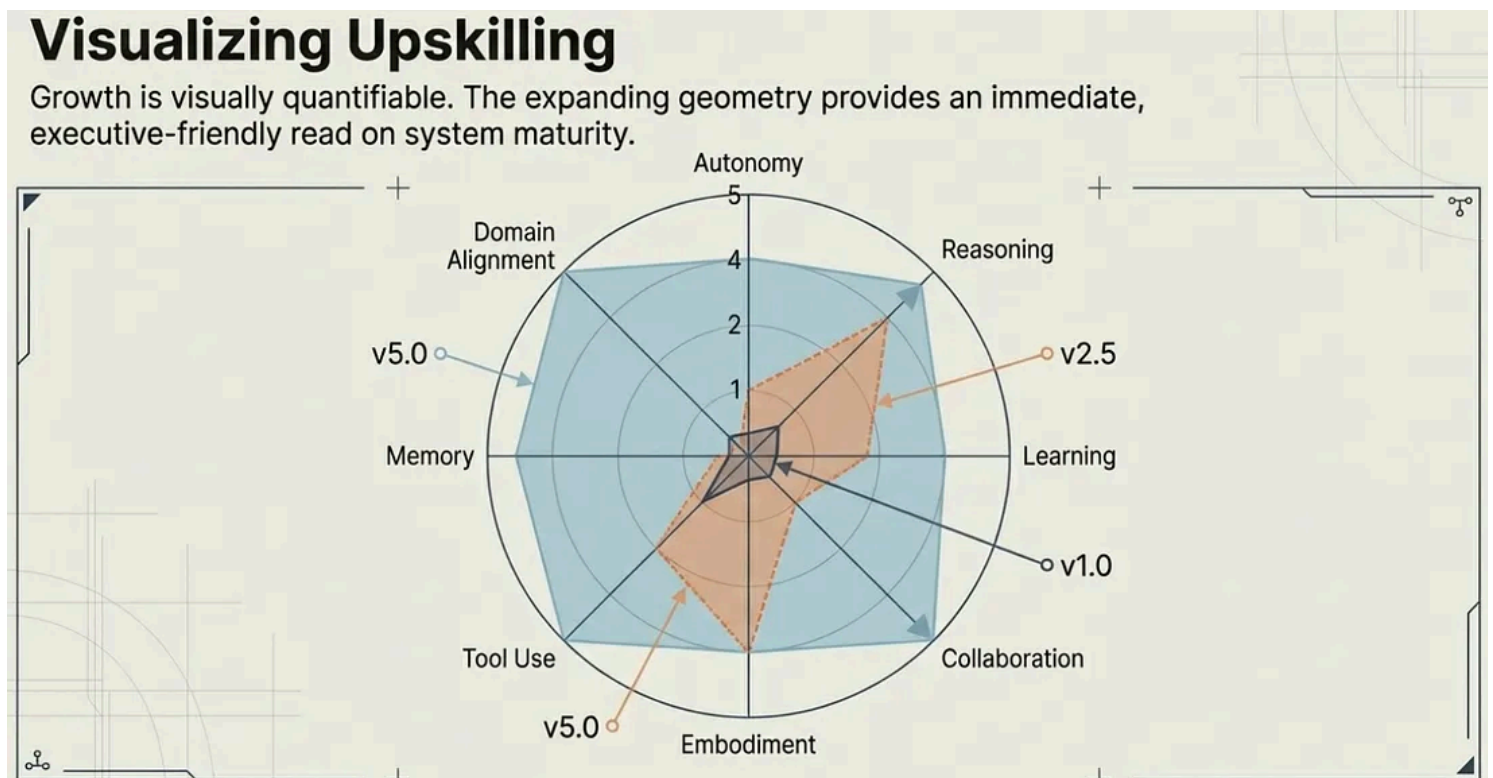
Example: *Alice*, an enterprise digital assistant.

- **v1.0** — Reactive chatbot: answers questions but does not execute work (Autonomy: 1, Reasoning :1, Tool Use: 0).
- **v2.5** — LLM-augmented assistant introduces retrieval and basic actions (Reasoning: 2, Memory: 2, Tool Use: 2) with human confirmation on side effects.
- **v4.0** — Goal-driven agent decomposes tasks and executes within policies (Autonomy: 4, Tool Use: 3) while achieving high Domain Alignment via permissioning, audit logs, and approval workflows.

- **v5.0 — Orchestrator:** coordinates specialized sub-agents, manages queues, and escalates exceptions (Reasoning: 5, Collaboration: 3) with stronger governance as blast radius increases.

This creates a governance trail that ensures **Domain Alignment** scales with **Autonomy** before deployment.

Once you can track an agent overtime, the next question becomes where it is allowed to operate. Different industries — and even different teams inside the same company — will weigh dimensions differently based on risk, regulation, and the cost of failure.

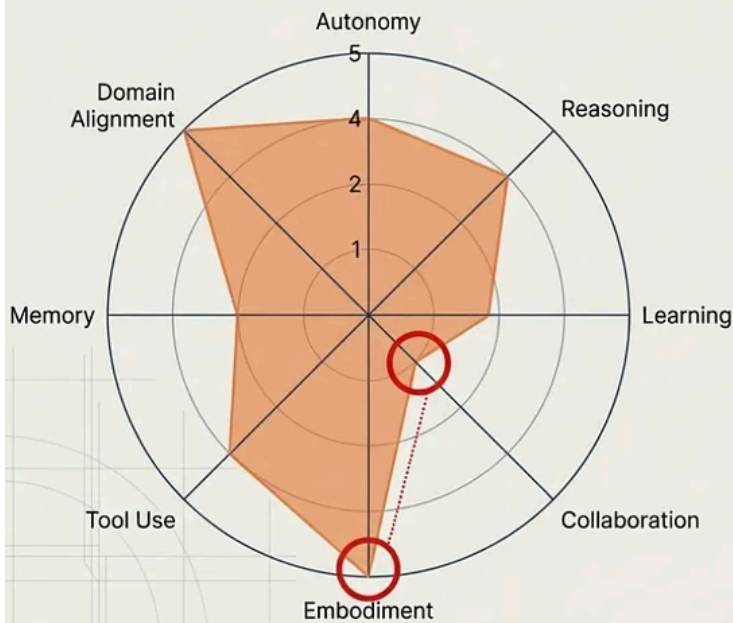


Agent Scorecard Example: Tesla Full Self-Driving (FSD)

Tesla's Full Self-Driving system provides a clear example of how the framework applies to a complex, embodied AI agent operating in a regulated domain.

Agent Scorecard: Tesla Full Self-Driving (FSD)

Target Domain: Automotive Transportation



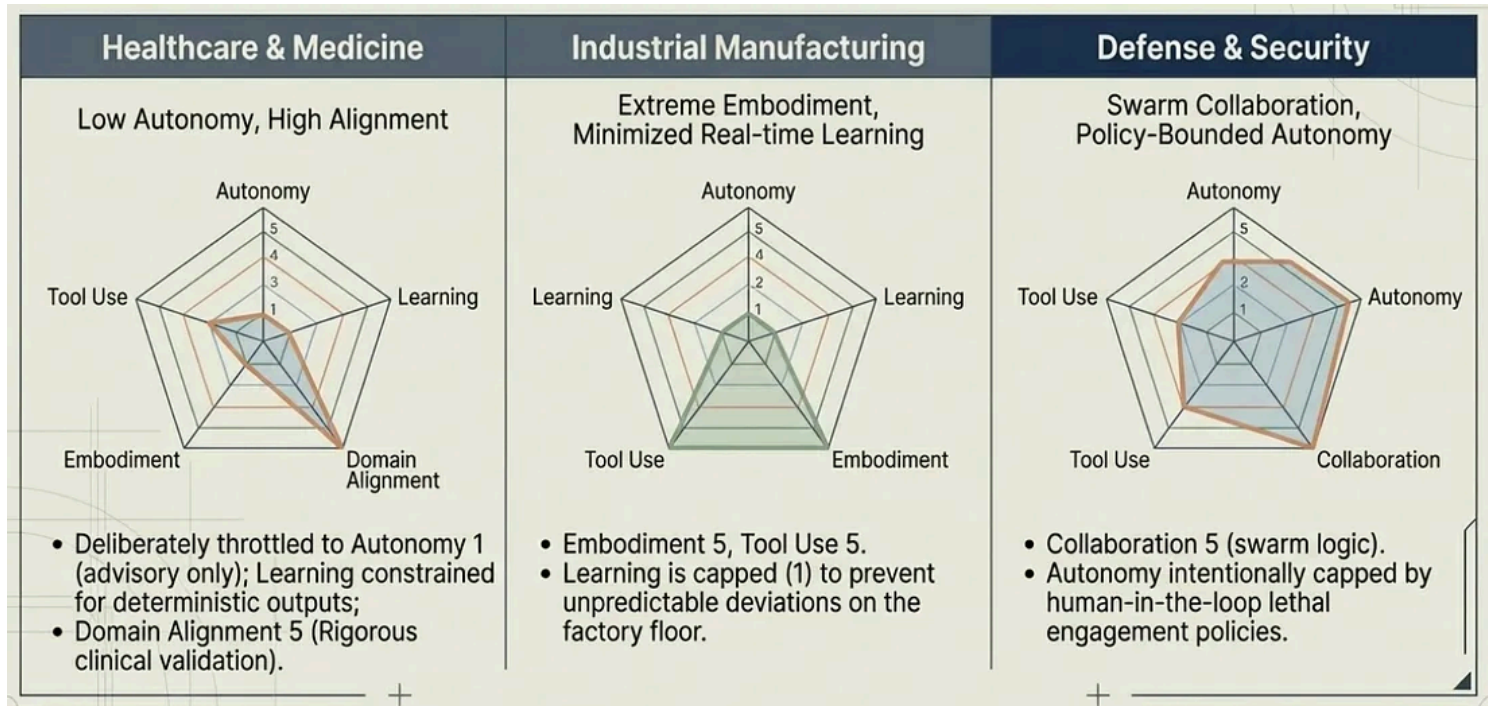
Autonomy [4/5]:	Handles diverse conditions, but requires constant human supervision (SAE L2).
Reasoning [3/5]:	Localized traffic navigation; no long-horizon strategic planning.
Learning [2/5]:	Over-the-air fleet updates; no real-time, on-device autonomous learning.
Collaboration [1/5]:	Isolated operation; lacks vehicle-to-vehicle (V2V) coordination.
Embodiment [5/5]:	Multi-ton real-time physical manipulation.
Tool Use [4/5]:	Advanced integration with vehicle control systems and sensors.

This scorecard demonstrates how the framework captures both **technical capability** and **regulatory constraints**, which are inseparable in safety-critical domains.

Dimension		Score	Analytical Notes
	Autonomy	★★★★★ 4/5	High. Operates the vehicle under diverse conditions but still requires constant, attentive human supervision (SAE Level 2).
	Reasoning & Planning	★★★★★ 3/5	Intermediate. Navigates localized traffic scenarios effectively but does not generate long-term strategic plans beyond immediate routing.
	Learning & Adaptation	★★★★★ 2/5	Limited. Continuous improvement occurs primarily via centralized, over-the-air fleet updates rather than real-time, on-device autonomous learning.
	Memory & Context	★★★★★ 3/5	Moderate. Retains short-term situational context and leverages persistent HD map data, but operates largely in the immediate present.
	Tool Use & Integration	★★★★★ 4/5	High. Deeply integrates an onboard suite of physical tools (cameras, radar, ultrasonic sensors) and digital navigation APIs.
	Collaboration	★★★★★ 1/5	Very Low. Functions as an isolated node; lacks vehicle-to-vehicle (V2V) coordination protocols.
	Embodiment	★★★★★ 5/5	Advanced. Direct, real-time physical manipulation of a multi-ton vehicle utilizing sophisticated robotics and environmental physics calculations.
	Domain Alignment	★★★★★ 4/5	High. Highly aligned with localized automotive safety standards, though falls short of full regulatory approval for unsupervised autonomous operation.
	Domain Alignment	★★★★★ 4/5	High. Highly aligned with localized automotive safety standards, though falls short of full regulatory approval for unsupervised autonomous operation.

Adapting the Framework to Different Industries

Different industries require different capability profiles. The same autonomy level can be acceptable in one context and unacceptable in another. In this model, **Domain Alignment** is the dimension that forces that reality into the scorecard by tying capability to constraints, evidence, and accountability.



Healthcare

In healthcare, the capability frontier is often set by compliance. Strong **Domain Alignment** (e.g., HIPAA controls, clinical safety processes, and traceability) is mandatory, while **Autonomy** is frequently capped, so systems remain advisory. Learning is typically controlled and audited to avoid drifting. The most valuable systems collaborate with clinicians surfacing evidence, uncertainty, and rationale rather than acting unilaterally.

Industrial Manufacturing

In industrial manufacturing, **Embodiment** and **Tool Use** dominate sensing, motion control, and integration with programmable logic controllers (PLCs), manufacturing execution systems (MES), and supervisory control and data acquisition (SCADA) systems. Many deployments intentionally minimize **Learning** to ensure repeatability, predictable cycle times, and safe failure

modes. Here, high capability often looks like tight constraints plus excellent monitoring not open-ended exploration.

Logistics & Transportation

In logistics and transportation, agents tend to score high on **Autonomy** and **Embodiment** within defined operational design domains, with heavy reliance on sensor fusion, telemetry, and fleet monitoring. **Domain Alignment** is again a gating factor (aviation, automotive, workplace safety), and successful systems are designed as socio-technical loops: autonomy does the routine work, while humans and processes handle exceptions.

Defense & Security

In defense and security, autonomy and collaboration matter because environments can be adversarial, disconnected, or time compressed. At the same time, Domain Alignment is shaped by rules of engagement, policy constraints, and rigorous assurance requirements — meaning autonomy is frequently capped by governance even when the underlying technology could do more. Multi-agent swarm behaviors elevate the importance of coordination protocols and fault containment.

Toward Standardized Agent Certification

As agents become core collaborators in enterprise systems, organizations need governance that is as rigorous as the software assurance practices we already apply to identity, data, and safety. The Agent Capability Framework supports that by giving cross-functional teams a shared language, enabling benchmarking across heterogeneous agents, and creating an explicit upgrade trail that can be reviewed in security, compliance, and architecture

boards. Over time, the same structure can support third-party certification — not as a one-time badge, but as a living profile tied to operational evidence.

In other words, once you can describe capability precisely, you can attach the right controls to it. The following workflow is a practical way to apply the model in a real organization.

How to Apply This Model (A Practical Workflow)

1. **Start with the operating domain.** Write down the constraints that actually matter: safety class, privacy rules, allowed actions, required auditability, and blast radius.
2. **Score from evidence, not aspiration.** Use logs, tests, red-team results, and reliability metrics to justify each dimension and record assumptions.
3. **Identify your bottleneck dimension.** Many failures come from a single weak link (for example, strong reasoning paired with weak tool reliability or weak domain alignment).
4. **Couple Autonomy to Alignment.** Treat rising autonomy as a trigger to strengthen guardrails, approvals, monitoring, and rollback before expanding scope.
5. **Track changes as a Capability Resume.** When you upgrade models, add tools, or introduce memory, re-score and document the delta so governance can keep pace.

Conclusion

The AI ecosystem is shifting from isolated tools to agentic systems woven into enterprise workflows and physical operations. Without a shared

taxonomy, organizations struggle to compare solutions, predict failure modes, and govern how agents evolve over time.

The **Agent Capability Maturity Model** provides that shared language. Use it to (1) score an agent across the eight dimensions, (2) justify scores with an evidence pack (tests, logs, red-team results, and audit trails), and (3) define governance triggers — especially that higher autonomy requires stronger domain alignment, monitoring, and rollback. Done well, the framework becomes a living “capability resume” you can review at each release gate, vendor evaluation, and production expansion.

Concrete next step: Pick one agent you already run (or are piloting). Score it quickly at a workshop with engineering, security, and product using the eight dimensions. Then choose one improvement that reduces real risk — tighten tool permissions, add audit logging, introduce an approval gate, or constrain the operating domain — and rescore after the change. That loop is how agent programs become measurable, governable, and scalable.

Further Reading

1. [JAFIMA: A Multi-Layered Agent Framework for Intelligent Mobile Agents in Java, JAFIMA | LinkedIn](#)
2. [Boston Dynamics' Spot Robot Gets Even More Capable With Enhanced Autonomy, Mobility — IEEE Spectrum](#)
3. [Amazon has launched our most advanced delivery drone yet — here's everything you need to know](#)
4. [What Tesla Autopilot And Full-Self Driving Can And Can't Do](#)
5. [AI Agent Taxonomy: Struggling with AI Agent Classification? Explore The Serious Insight's Focused Take on Agents. — Serious Insights](#)

6. [Agent Maturity Model | The Agentic Engineering Guide](#)
7. [Outshift | 5 Levels of agentic AI intelligence for enterprise use](#)
8. [The Five Levels of Agentic AI Maturity — Cognition Consulting Group](#)
9. [Salesforce releases agentic maturity model for enterprises — Salesforce](#)
10. [Anchoring Autonomy in the Hybrid Cloud: The Agent Maturity Model \(AMM\) | LinkedIn](#)

Framework

Maturity Model

Ai Agents

Acmm

Agentic Ai

Some rights reserved 



Written by Suresh Chande

0 followers · 6 following

Edit profile

Principal Product Manager, Agent Momentum Team @ IDEAS (Insights, Data, Employee Experience, Agents & Science), Data Platform Group, E&D, Microsoft.

